Dein Ablauf auf einen Blick

Etwa 10–12 Minuten · Erst zeigen, dann erklären

Du beginnst mit dem laufenden System und bestätigst einen Alarm. Anschließend erklärst du die Architektur entlang dieses Vorgangs.

1. **0:00–2:00 · Live zeigen.** Messwerte laufen lassen, einen Alarm im Leitstand bestätigen und das Ergebnis im Monitor zeigen.
2. **2:00–3:00 · Gesamtsystem zeigen.** Auf der Architekturkarte alle Komponenten und ihre Verbindungen benennen.
3. **3:00–5:00 · Sensor und Relay erklären.** Wie wird aus einem Messwert ein gespeicherter Alarm?
4. **5:00–7:00 · Browser und Bestätigung erklären.** Wer besitzt welchen Zustand? Wann gilt eine Bestätigung als gespeichert?
5. **7:00–10:00 · Codeentscheidungen zeigen.** Wenige passende Ausschnitte statt einer Tour durch jede Datei.
6. **10:00–12:00 · Grenzen und Fragen.** Simulator und Hysterese als gezielte Vertiefung bereithalten.

Dein roter Faden: Messwert → Erkennung → Speicherung → Anzeige → Bestätigung. Nach jedem Teil sagen, was zum nächsten Teil weitergegeben wird.

1 · Mit den echten Apps starten

0:00–2:00 · Das System zunächst laufen lassen

ZEIGEN

Live Monitor und Control Room in zwei Tabs öffnen. Lokal mit **demo / demo** anmelden. Einen offenen Alarm mit seiner ID bereithalten. Die Messwerte zunächst kurz laufen lassen.

SAGEN

„Ich zeige zuerst, was die Anwendung macht. Hier kommen laufend Temperaturwerte an. Der Monitor zeigt die Messwerte, die Einstellungen und die erkannten Alarme. Im Control Room kann ein Operator auf diese Alarme reagieren.“

Einen Alarm bestätigen

ZEIGEN

Im Control Room bei einem offenen Alarm auf **Acknowledge** klicken. Danach im Monitor denselben Alarm anhand seiner ID zeigen.

SAGEN

„Ich bestätige diesen Alarm. Die Änderung wird vom Server gespeichert und an die verbundenen Oberflächen verteilt. Beide Ansichten zeigen danach dieselbe Bestätigung. Die Messwerte laufen währenddessen weiter.“

Zur Erklärung überleiten

„Jetzt schauen wir uns an, welche Teile daran beteiligt sind und wie dieser Vorgang technisch durch das System läuft.“

Die echten Apps haben englische Beschriftungen. Der lokale Sensor liefert 2 Messwerte pro Sekunde. Falls kein offener Alarm vorhanden ist, nutze den Simulator als angekündigten Ersatz; warte nicht lange auf zufällige Werte.

2 · Das ganze System zeigen

2:00–3:00 · Architekturleitfaden → Systemübersicht

> **ZEIGEN**
>
> Den Architekturleitfaden unter **localhost:5175** öffnen, **DE** wählen und die gesamte Karte zeigen. Noch keine Dateien öffnen.

> **SAGEN**
>
> „Das System besteht aus einem Sensordienst, einem Relay mit SQLite und zwei Browseranwendungen. Ich gehe die Teile jetzt in der Reihenfolge durch, in der wir den Alarm gerade gesehen haben.“

Auf die Teile zeigen

1. **Sensor:** erzeugt Messwerte und erkennt Übergänge über die Hysteresegrenzen.
2. **Relay:** nimmt Nachrichten entgegen, prüft sie und verteilt Änderungen.
3. **SQLite:** speichert Alarme, Bestätigungen und Einstellungen dauerhaft.
4. **Live-Monitor:** zeigt Messwerte, Diagramm, Alarme und Einstellungen.
5. **Leitstand:** konzentriert sich auf die Alarmliste und die Operatoraktion.

> **SAGEN**
>
> „Die Erkennung hängt nicht von einem offenen Browser ab. Der Sensor erkennt den Übergang. Der Relay verwaltet die gespeicherten gemeinsamen Datensätze. Die Browser zeigen jeweils eine lokale Kopie davon.“

Übergang: „Beginnen wir dort, wo der Messwert entsteht: beim Sensor.“

3 · Sensor: vom Wert zum Alarm

3:00–4:00 · Auf Sensor und ausgehende Verbindung zeigen

> **SAGEN**
>
> „Die aktuelle Datenquelle simuliert Temperaturen. Jeder Messwert hat einen Zeitpunkt und einen Wert. Der Agent verbindet diese Quelle mit der Erkennung und dem Versand. Ein Messwert und ein Alarm sind unterschiedliche Nachrichten.“

Warum Hysterese?

Bei einem Schwellwert von **25°C** und einer Hysteresebreite von **0,6°C** liegen die Grenzen bei **24,7°C und 25,3°C**.

Von unten kommend entsteht erst oberhalb von 25,3°C ein Aufwärtsalarm. Danach entsteht erst unterhalb von 24,7°C ein Abwärtsalarm. Gleichheit an einer Grenze löst keinen Wechsel aus.

> **SAGEN**
>
> „Damit erzeugt ein leichtes Schwanken um 25 Grad nicht ständig neue Alarme. Der erste Messwert legt nur die Ausgangsseite fest. Ein Alarm beschreibt einen späteren Übergang.“

Was wird weitergegeben?

Normale Messwerte werden als laufender Datenstrom übertragen. Erkannte Alarme erhalten eine UUID und werden für die Zustellung im Outbox-Modell gehalten.

Entscheidung: Die Erkennung ist eine reine Zustandsfunktion ohne Netzwerkzugriff oder Uhr. Dadurch lassen sich Grenzwerte und Messfolgen direkt testen.

4 · Relay: speichern und verteilen

4:00–5:00 · Auf Relay und SQLite zeigen

> **SAGEN**
>
> „Der Relay ist die gemeinsame Vermittlungsstelle. Er prüft Nachrichten anhand des Protokolls, speichert Alarme und verteilt neue Datensätze an die Browser.“

Versand ist noch keine Speicherung

Ein erfolgreicher Socket-Schreibvorgang beweist nicht, dass der Alarm in SQLite steht. Der Sensor hält den Alarm deshalb ausstehend, bis der Relay dessen Annahme mit **accepted** bestätigt.

> **SAGEN**
>
> „Fehlt diese Annahme, kann der Sensor den Alarm mit derselben UUID erneut senden. Wenn der Relay ihn bereits gespeichert hat, bestätigt er ihn erneut, ohne einen zweiten neuen Alarm an die Browser zu verteilen.“

Was bleibt erhalten?

SQLite enthält Alarme, Operatorbestätigungen und Einstellungen. Gewöhnliche Messwerte liegen nur in begrenzten Puffern; eine vollständige Temperaturhistorie wird nicht gespeichert.

Grenze offen nennen: Die Sensor-Outbox ist begrenzt und liegt im Arbeitsspeicher. Prozessabbruch oder Überlauf können Alarme verlieren lassen. Die Wiederholungslogik ist keine uneingeschränkte Zustellgarantie.

Übergang: „Der Datensatz ist jetzt gespeichert. Wie kommt er in die beiden Oberflächen?“

5 · Oberflächen und Bestätigung

5:00–7:00 · Monitor, Leitstand und Rückweg zeigen

Live-Monitor

> **SAGEN**
>
> „Der Monitor übersetzt Relay-Nachrichten in Zustandsänderungen. Redux Toolkit verwaltet bestätigte Einstellungen, Sensorinformationen und Alarme. Zähler und Ansichten werden daraus abgeleitet.“

Messwerte haben einen eigenen begrenzten Store. Ein Formularentwurf bleibt lokal, bis der Benutzer ihn übernimmt. Der aktive gemeinsame Wert kommt über den Relay zurück.

Leitstand

Die kleinere Alarmliste verwendet **useReducer**. Sie benötigt dafür keinen eigenen Redux-Store. Beide Apps zeigen dennoch dieselben serverseitigen Datensätze.

Zurück zur Bestätigung vom Anfang

> **SAGEN**
>
> „Der Klick sendet eine Anfrage mit der Alarm-ID. Der Relay nimmt die Identität des Operators aus der angemeldeten Sitzung, speichert die erste Bestätigung und verteilt das Ergebnis. Erst die Rückmeldung bestätigt den gemeinsamen Zustand.“

Zwei Begriffe unterscheiden: accepted bedeutet: Der Relay hat den Alarm angenommen. Die Operatorbestätigung bedeutet: Eine Person hat auf den Alarm reagiert.

Bei zwei gleichzeitigen Bestätigungen bleibt die erste gespeicherte Bestätigung erhalten. Eine Bestätigung beendet nicht den Messwertstrom und setzt den Sensor nicht zurück.

6 · Die wichtigsten Codeausschnitte

7:00–10:00 · Code-Explorer, dann Abschluss

> **ZEIGEN**
>
> Im Code-Explorer die Projektgruppen nutzen. Nur die genannten Stellen zeigen; den Code nicht Zeile für Zeile vorlesen.

Sensor → Grenzwertdetektor

domain/crossing.ts: die strikten Vergleiche zeigen. „Vorheriger Zustand, Messwert und Konfiguration bestimmen das Ergebnis. Ich kann die Fachlogik ohne Socket oder Timer testen.“ Quelle und Agent bei Bedarf vorher kurz zeigen.

Sensor → Übertragungs-Outbox; Relay → Alarm weiterleiten

outbox.ts / routing.ts: ausstehende Alarme, **accept**, **expire** und den Zweig für bekannte IDs zeigen. „Wiederholung behält die UUID; der Relay verhindert einen zweiten neuen Alarmdatensatz.“

Live-Monitor → Redux-Zustand und Reducer

store/alertsSlice.ts: Zustandsbeispiel und Bestätigungsupdate zeigen. „IDs bestimmen die Reihenfolge, entities enthält die Datensätze. Immer erlaubt Änderungen am Entwurf und erzeugt den unveränderlichen Folgezustand.“

Abschluss

> **SAGEN**
>
> „Die zentralen Entscheidungen sind eine testbare Erkennung, eine ausdrückliche Zustellbestätigung und klare Zustandsverantwortung. Als nächste Verbesserungen würde ich die Sensor-Outbox dauerhaft speichern und die Wiederherstellung nach Verbindungsabbrüchen erweitern.“

Reserve · Gezielt im Simulator zeigen

Nur bei Bedarf · Exakte Abläufe statt Zufallswerte

Übergang und Bestätigung

> **ZEIGEN**
>
> Live-Simulator → **Übergänge in beide Richtungen** beziehungsweise das Szenario **Cross both directions** → Neustart. Mit **Schritt** manuell fortfahren.

Schritte 1–9: 24°C, 25,1°C und genau 25,3°C ergeben noch keinen Alarm. **Schritt 10:** 25,4°C erzeugt den Alarm. **11:** Relay speichert und nimmt an. **12:** Browser zeigen den Alarm.

Danach **Bestätigen** und dreimal **Schritt**: speichern → verteilen → in beiden Ansichten übernehmen. Routen bleiben zwischen den Klicks markiert.

Hystereseband anpassen

Szenario **Hystereseband anpassen** → **Neustart**. Die folgenden Zählungen gelten ohne zusätzliche Bestätigung:

- **1–6:** zwei Messwerte, noch kein Alarm.
- **7–9:** Bandbreite von 0,6 auf 0,2°C. Neue Grenzen: 24,9 und 25,1°C. Die Änderung selbst erzeugt keinen Alarm.
- **10–12:** stille Neueinordnung, kein Übergang.
- **13–15:** 24,85°C erzeugt einen Abwärtsalarm.
- **16–18:** 25,15°C erzeugt einen Aufwärtsalarm.

„Das engere Band reagiert auf kleinere Änderungen. Beide späteren Werte lagen noch innerhalb des alten Bands.“

Reserve · Kurze Antworten

Für Rückfragen nach der Präsentation

Warum Redux und nicht Zustand?

Redux Toolkit macht Aktionen, normalisierte Datensätze und abgeleitete Ansichten gut nachvollziehbar. Zustand wäre ebenfalls möglich. Die Entscheidung passt zum Zustandsmodell des Monitors; sie ist keine technische Notwendigkeit.

Warum ein eigener Store für Messwerte?

Messwerte sind vorübergehend und begrenzt. Diagramm und Anzeige abonnieren sie über useSyncExternalStore. Redux könnte die Rate ebenfalls verarbeiten; entscheidend sind Lebensdauer und Verantwortung.

Stellt Reconnect alles wieder her?

Nein. Die Begrüßungsnachricht enthält höchstens 100 aktuelle Alarme. Ein älterer, lokal vorhandener Alarm kann eine Bestätigung während einer Unterbrechung verpassen. Ein dauerhafter Änderungsverlauf mit Wiederholung wäre eine Erweiterung.

Wie würdest du neue Funktionen ergänzen?

Zuerst Verhalten und Zuständigkeit klären, dann Datenfluss und Tests festlegen. Mehrere Sensoren brauchen getrennte Identitäten und Zustände. Alarmhistorie benötigt eine autorisierte, paginierte Abfrage. Vollständige Messhistorie braucht zusätzliche Speicherung.

Was ist nur simuliert?

Die Temperaturquelle erzeugt aktuell künstliche Werte. Die echten Apps verwenden dennoch reale Netzwerkverbindungen und Speicherung. Der Architektur-Simulator ist ein separates Erklärmodell und steuert diese Verbindungen nicht.